Final Project Design Phase

Computer Science - Week 11

Oct 3, 2026 - Version 0.0.4

For complete instructions and project ideas, refer to the assignment in Canvas. Specifically, review the final project requirements before you continue below. If you have not reviewed the project requirements in Canvas (you might have to go to the modules view and check the current or past week's content), you will not be able to properly complete the parts below. In addition, it may be helpful to review the information on this Drafter documentation page.

0) Project Planning and Preparation

Successful programmers spend time planning before they begin coding. *After* reviewing the project requirements, answer the following questions to help guide your design and development process.

A. What concepts from class or previous assignments might help you complete this project? Which programming concepts (loops, conditionals, lists, classes, etc.) do you expect to use? What programming skills do you want to improve while completing this project?

From our previous BlockPy and Drafter assignments, creating @dataclass structures, building @route functions that pass and update State, and writing assert_equal unit tests will help me complete this project. I expect to use dataclasses (State and Expense), lists of dataclass instances (list[Expense]), for loops to add up and filter expenses, if/elif/else conditionals to check if spending is over budget or if user inputs are valid, and string to float type conversions. While completing this project, I want to get better at keeping State consistent across multiple pages and building code that won't crash.

B. Which parts of the project do you expect to be the easiest for you? The hardest? Why?

The easiest part for me will be setting up the Expense and State dataclasses. We have practiced that a lot in class. The hardest part will be handling the user inputs from TextBox, SelectBox, and CheckBox across 7 different routes without crashing. Especially converting the cost from a string to a float safely. Writing thorough assert_equal unit tests for every single route function will also be time consuming and challenging.

C. Even though you have not yet designed the final project, estimate which tasks will likely require the most time and effort? Which features or components should be implemented first?

Writing the route logic to add, filter, and calculate expenses will likely take the most time (around 3 to 4 hours total), while setting up the dataclasses and basic page navigation should take about 1 to 2 hours. I should implement the Expense and State dataclasses first, followed by the index home route and the save_budget route, and then build the add_expense route and for loop summary calculations before adding the category filter and reset routes.

D. What distractions might interfere with your work and how will you minimize them? What resources will you need available while working? What questions, if any, do you still have about the assignment requirements?

Common distractions like my phone, social media, and background noise in my dorm could slow me down, so I plan to put my phone on Do Not Disturb and work in the library during set 1-to-2 hour blocks. While working, I will keep the official Drafter documentation, our CISC106 lecture notes and past BlockPy labs, and TA office hours schedule open. One question I still want to double-check as I start coding is the cleanest way in Drafter to handle a user typing non-numeric text into a TextBox meant for a float cost without failing unit tests.

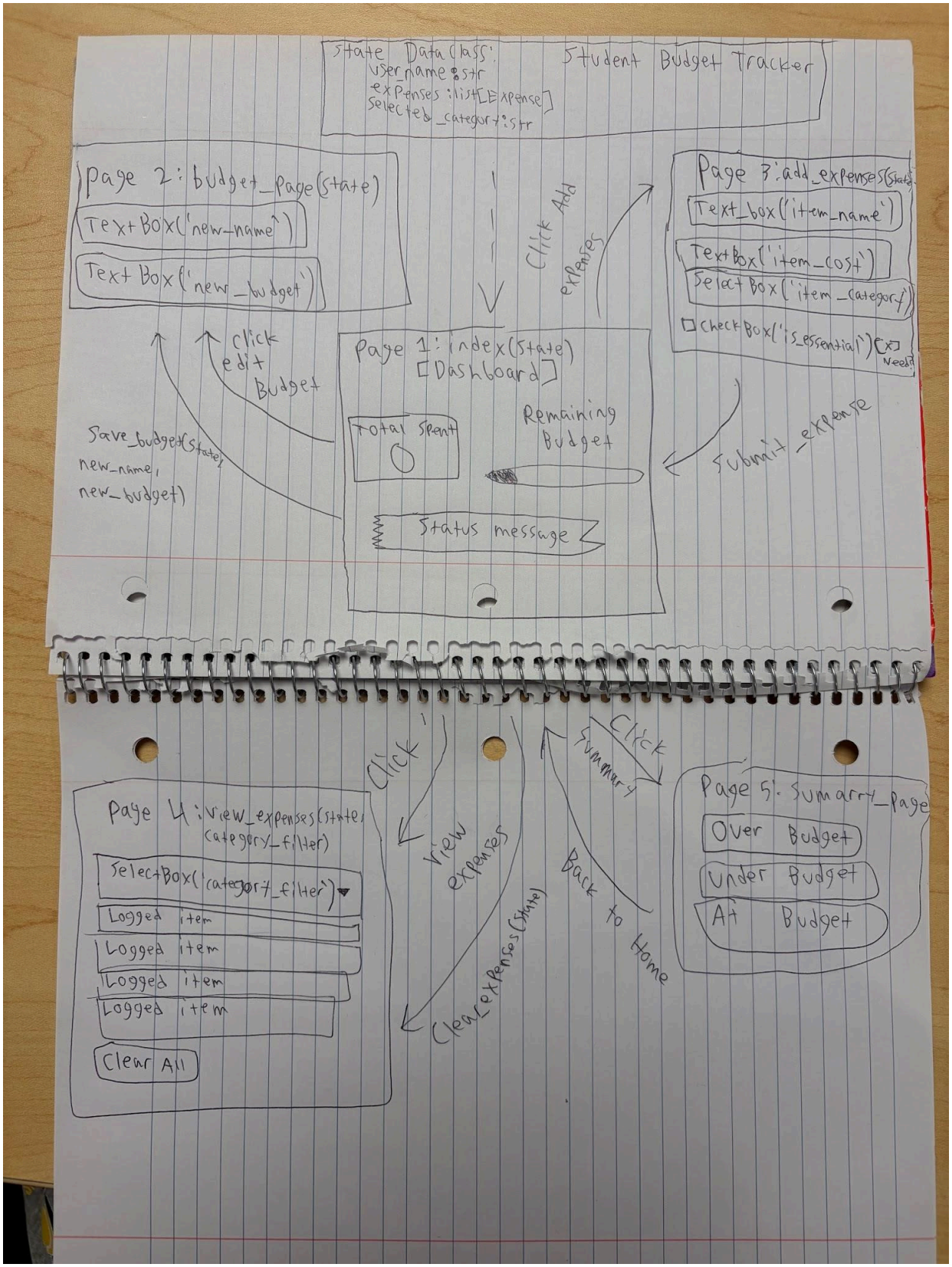
1) Site Diagram

Create a mock-up of your planned website. We suggest you make the mockup on paper, although you can also use software like PowerPoint or Slides to make a mockup. There is other design software that could also be used, but we strongly encourage you to keep it simple. Do not spend more than 15-30 minutes detailing the mock-up. Your goal should be to make a very quick and dirty diagram, to show that you have some kind of idea.

We recommend drawing each page as a rectangle, and laying out the text, buttons, and other components inside. Then, use arrows to indicate links between pages. Label each page the way you will name each function, and then put labels on the arrows to indicate what the button links. Also write down any parameters that should come into the pages.

In addition to the pages and routes, write down what fields your `State` will have, both their names and types. Think clearly about what data will need to be persistent across the entire website.

Add photo(s) or screenshot(s) of your website mock-up and labeled diagram on the next page.


State Dataclass:
user_name: str
expenses: list[Expense]
selected_category: str
Student Budget Tracker
Page 2: budget_page(state)
TextBox('new_name')
TextBox('new_budget')
click edit Budget
Save_budget(state, new_name, new_budget)
Page 1: index(state)
[Dashboard]
Total Spent
Remaining Budget
Status message
Click Add expenses
Page 3: add_expenses(state)
Text_box('item_name')
TextBox('item_cost')
SelectBox('item_category')
CheckBox('is_essential')
Submit_expense
Page 4: view_expenses(state, category_filter)
SelectBox('category_filter')
Logged item
Logged item
Logged item
Logged item
Clear All
Click view expenses
Clear_expenses(state)
Click Summary
Back to Home
Page 5: Summary_page
Over Budget
Under Budget
At Budget

2) Implementing Requirements

This table should serve as a draft for how you will implement the requirements into your final project. You may change your ideas later on, but your final site must fulfill all the requirements.

Requirement	**Implementation**
Purpose/Functionality A 1-2 sentence description of the purpose or functionality of your site	This web application helps college students set a monthly spending budget, log daily expenses by category and whether each item is an essential need or a want, and track how much money they have left so they do not overspend.
State Dataclass Attributes Minimum of 4 attributes, one of which must be a list/dict of instances of your second dataclass. Briefly describe each attribute, its type, and how it will be meaningfully modified throughout the site.	1. **user_name: str** — stores the student's name; modified in the **save_budget** route when the user updates their profile/budget settings. 2. **monthly_budget: float** — stores the monthly spending limit in dollars; modified in **save_budget** when the user sets or changes their budget. 3. **expenses: list[Expense]** — stores a list of **Expense** dataclass instances; modified in **submit_expense** (which appends a new **Expense**) and **clear_expenses** (which resets the list to empty). 4. **selected_category: str** — stores the active category filter ('All', 'Food', 'School', 'Rent', or 'Fun'); modified in **view_expenses** when the user filters their expense list. 5. **status_message: str** — stores a feedback or budget warning banner; modified across **save_budget**, **submit_expense**, and **clear_expenses** to reflect the latest action and budget status.
Second Dataclass Your second dataclass, including its attributes, their types, and how they will be meaningfully modified (if they are modified)	**Expense** dataclass with 4 attributes: 1. **name: str** — the name or short description of the purchase (e.g., 'Dining Hall Coffee' or 'Chem Lab Manual'). 2. **category: str** — the spending category ('Food', 'School', 'Rent', or 'Fun'). 3. **cost: float** — the dollar price of the expense. 4. **is_essential: bool** — **True** if the purchase is an essential need, or **False** if it is a discretionary want. New **Expense** instances are created and added to **state.expenses** in the **submit_expense** route whenever a user logs an item.

Routes (minimum 7) The names of your route functions	1. **index(state: State) -> Page** (Home dashboard page) 2. **budget_page(state: State) -> Page** (Displays the form to edit name and monthly budget) 3. **save_budget(state: State, new_name: str, new_budget: str) -> Page** (Processes budget form inputs, modifies state.user_name, state.monthly_budget, and state.status_message, and returns the dashboard) 4. **add_expense_page(state: State) -> Page** (Displays the form to log a new expense) 5. **submit_expense(state: State, item_name: str, item_cost: str, item_category: str, is_essential: bool) -> Page** (Creates a new Expense, appends it to state.expenses, updates state.status_message, and goes to the expense list) 6. **view_expenses(state: State, category_filter: str) -> Page** (Updates state.selected_category and displays the filtered list of expenses) 7. **summary_page(state: State) -> Page** (Calculates and displays the Needs vs. Wants breakdown and budget status warnings) 8. **clear_expenses(state: State) -> Page** (Resets state.expenses to an empty list and updates state.status_message)
Pages (minimum 5) A brief description of the pages in your site	1. **Home Dashboard Page:** Shows the user's name, monthly budget, total spent so far, remaining balance, status message, and navigation buttons to all other pages. 2. **Set/Edit Budget Page:** Provides TextBox inputs for the student to enter or update their name and monthly dollar budget. 3. **Add Expense Page:** Provides TextBox, SelectBox, and CheckBox inputs for logging a new purchase's name, cost, category, and whether it was an essential need. 4. **View & Filter Expenses Page:** Lists logged expenses and includes a SelectBox to filter the displayed items by category (All, Food, School, Rent, Fun) plus a button to clear expenses. 5. **Spending Summary & Breakdown Page:** Shows how much money was spent on essential Needs vs. discretionary Wants, the percentage of the monthly budget used, and a warning message if spending is over 80% or 100% of the limit.
User Input (minimum 3) Description of user input components (TextBox, TextArea, CheckBox, or SelectBox) and	1. **TextBox('new_budget')** on the Set Budget page (and **TextBox('item_name')** / **TextBox('item_cost')** on the Add Expense page) so the user can type in their budget amount, item name, and dollar cost. 2. **SelectBox('item_category', ['Food', 'School', 'Rent', 'Fun'])** on the Add Expense page (and **SelectBox('category_filter', ['All', 'Food', 'School', 'Rent',**

their purposes For more information on these input types, see Drafter Documentation.	**'Fun'])** on the View Expenses page) so the user can pick an expense category or filter by category. 3. **CheckBox('is_essential')** on the Add Expense page so the user can mark whether an expense is an essential need (True) or a non-essential want (False).
If Statements (minimum 3) Brief description of what will be evaluated in your if statements and how they will meaningfully change the logic of the site	1. **Budget Limit Warning Check:** In **summary_page** and **submit_expense**, an **if/elif/else** statement checks if **total_spent > state.monthly_budget** (displays an 'Over Budget!' warning), **elif total_spent >= 0.8 * state.monthly_budget** (displays a 'Close to Limit (80%+)' warning), **else** (displays 'Under Budget'). 2. **Category Filter Check:** In **view_expenses**, an **if** statement inside the expense loop checks if **state.selected_category == 'All' or expense.category == state.selected_category** to decide which expenses to show on the page. 3. **Essential Need vs. Want Check:** In **summary_page**, an **if/else** statement checks **if expense.is_essential:** to add the expense's cost to **essential_total** versus **wants_total** so the student can see how much they are spending on needs vs. wants.
Loop (minimum 1) This loop must iterate over the list/dict of dataclass instances in your State attribute. Description of this loop and why it is meaningful to the site	A **for** loop (**for expense in state.expenses:**) iterates over the **Expense** dataclass instances stored in **state.expenses**. It is used to add up **expense.cost** to compute total monthly spending, calculate the Needs vs. Wants subtotals (**if expense.is_essential**), and build the filtered list of formatted expense strings to display on the View Expenses and Summary pages.

LLM Conversation

If you used an LLM in any part of this work, copy and paste every part of that conversation below. Otherwise, you may be given a zero for the assignment. Obviously, these words will not all be yours, and this is the exception to the requirement at the top of this assignment that all responses be in your own words. If you did not use an LLM to assist you in this assignment, simply say so below.

User: For Part 1 of my CISC106 final project design, I'm planning out my Student Monthly Budget & Expense Tracker in Drafter. We need at least 5 pages, 7 routes, a State dataclass with at least 4 attributes (including a list of a second dataclass), 3 input types, 3 if statements, and 1 loop. Right now I have an Expense dataclass with name, category, cost, and is_essential. Can you help me brainstorm what my 7 routes and 5 pages should be so they connect cleanly?

AI: Having an Expense dataclass with name: str, category: str, cost: float, and is_essential: bool is a great foundation! Here is a clean way to organize your 5 pages and 8 routes so every page has a clear job and modifies State meaningfully:

1. Home Dashboard Page (index(state: State) -> Page): Shows the user's name, monthly budget, total spent, remaining balance, and navigation buttons.

2. Budget Setup Page (budget_page(state: State) -> Page and save_budget(state: State, new_name: str, new_budget: str) -> Page): Lets the student enter their name and monthly budget into TextBox inputs and saves them to State.

3. Add Expense Page (add_expense_page(state: State) -> Page and submit_expense(state: State, item_name: str, item_cost: str, item_category: str, is_essential: bool) -> Page): Uses TextBox, SelectBox, and CheckBox inputs to create a new Expense and append it to state.expenses.

4. View & Filter Expenses Page (view_expenses(state: State, category_filter: str) -> Page and clear_expenses(state: State) -> Page): Displays logged expenses filtered by category and includes a button to clear all expenses.

5. Spending Summary Page (summary_page(state: State) -> Page): Shows how much was spent on essential Needs vs. Wants and displays a budget warning message.

User: That makes sense. For my State dataclass, if I have user_name: str, monthly_budget: float, expenses: list[Expense], selected_category: str, and status_message: str, how should I use my 1 for loop and 3 if statements across those routes?

AI: You can use your for loop (for expense in state.expenses:) to iterate through every Expense object in state.expenses, and combine it with your 3 if statements like this:

1. Category Filter Check (in view_expenses)

2. Essential Need vs. Want Check (in summary_page)

3. Budget Limit Warning Check (in summary_page and submit_expense)

Final Submission

When you are finished, download this file as a Word Document (docx) and upload the file to the appropriate assignment on Canvas.